Chương 1 (Phần 2): Giới thiệu về an toàn phần mềm

1.4 Kiểm thử & kiểm chứng (II) – 1.5 Cấu trúc Kripke – 1.6 Tổng kết

ThS. Vũ Quang Dũng

Khoa Hệ thống thông tin – Trường Đại học Phenikaa

CSE703093 – An toàn phần mềm

Nhắc lại Phần 1

- 1.1 Khái niệm an toàn phần mềm: CIA, asset/threat/vulnerability/risk, STRIDE
- 1.2 Các vấn đề an toàn phần mềm: CWE/CVE, buffer overflow, injection, race condition...
- 1.3 Kiểm thử & kiểm chứng (I): V&V, static/dynamic, V-model, black-box techniques

Phần 2 sẽ đi sâu vào kỹ thuật white-box, cấu trúc hình thức (Kripke), và tổng kết chương.

Nội dung Phần 2

1. 1.4 Kiểm thử và kiểm chứng phần mềm (Phần II)
2. 1.5 Cấu trúc Kripke (Kripke Structure)
3. 1.6 Tổng kết Chương 1

Các kỹ thuật thiết kế test bổ sung

Ngoài equivalence partitioning và boundary value analysis đã học, còn có:

- **Decision Table Testing**: kiểm thử dựa trên tổ hợp điều kiện logic
- **State Transition Testing**: kiểm thử dựa trên các trạng thái và chuyển đổi trạng thái
- **Use Case Testing**: kiểm thử dựa trên kịch bản sử dụng thực tế
- **Experience-based**: exploratory testing, error guessing (dựa kinh nghiệm)

Decision Table Testing – Ví dụ

Quy tắc giảm giá: giảm 10% nếu là thành viên *và* đơn hàng $>$ 500k.

	TC1	TC2	TC3	TC4
Là thành viên?	Đ	Đ	S	S
Đơn $>$ 500k?	Đ	S	Đ	S
Giảm giá 10%?	Có	Không	Không	Không

Decision table đảm bảo mọi tổ hợp điều kiện logic quan trọng đều được kiểm thử.

State Transition Testing – Ví dụ

Áp dụng cho mô hình trạng thái đăng nhập đã xây dựng ở module Python `ch1_models`:

- Test case theo trạng thái: `idle` → `authenticated` (đăng nhập đúng)
- Test case theo trạng thái: `idle` → `locked` (sai quá số lần)
- Test case chuyển đổi không hợp lệ: từ `locked` có được phép chuyển thẳng sang `authenticated` không?

Đây chính là cầu nối trực tiếp tới Kripke Structure ở mục 1.5.

Định nghĩa

Use Case Testing: thiết kế test case dựa trên các kịch bản sử dụng thực tế của hệ thống (main flow và các alternative/exception flow).

Ví dụ

Use case “Đăng nhập”: main flow (nhập đúng thông tin), alternative flow (quên mật khẩu), exception flow (nhập sai 5 lần liên tiếp $\rightarrow$ khóa tài khoản).

Experience-based Testing

- **Error Guessing**: dựa vào kinh nghiệm để đoán các lỗi thường gặp (input rỗng, ký tự đặc biệt, số âm...)
- **Exploratory Testing**: vừa thiết kế vừa thực thi test, học hỏi về hệ thống trong quá trình kiểm thử, không theo kịch bản cố định trước
- Hữu ích để bổ sung cho các kỹ thuật có hệ thống, đặc biệt khi tìm lỗi bảo mật “phi tiêu chuẩn”

White-box Testing: Control Flow Graph (CFG)

Định nghĩa

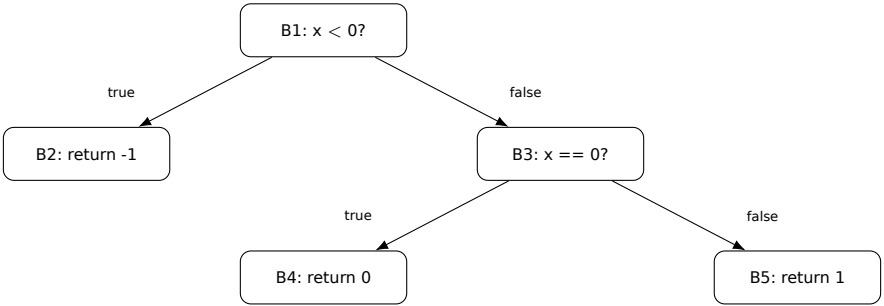
Control Flow Graph: biểu diễn đồ thị các đường thực thi có thể có của chương trình, với các nút là khối lệnh và cạnh là luồng điều khiển.

CFG là cơ sở để tính các độ đo coverage (statement, branch, path) và độ phức tạp (cyclomatic complexity).

Ví dụ mã nguồn cho CFG

```
int classify(int x) {
    if (x < 0) {           // B1
        return -1;          // B2
    } else if (x == 0) {   // B3
        return 0;           // B4
    } else {
        return 1;           // B5
    }
}
```

CFG tương ứng



Statement Coverage

Định nghĩa

Statement Coverage = (Số câu lệnh đã thực thi) / (Tổng số câu lệnh) $\times$ 100%

Ví dụ

Với hàm `classify`, test case x=`-1` và x=`0` đã bao phủ B1, B2, B3, B4 – còn thiếu B5 (x=1). Statement coverage = 4/5 = 80%.

Branch (Decision) Coverage

Định nghĩa

Branch Coverage = (Số nhánh quyết định đã thực thi) / (Tổng số nhánh) $\times$ 100%

Branch coverage yêu cầu *cả hai* nhánh true/false của mỗi điều kiện đều được thực thi ít nhất một lần – chặt chẽ hơn statement coverage.

Lưu ý

100% statement coverage **không đảm bảo** 100% branch coverage (ví dụ: câu lệnh `if` không có `else` có thể đạt statement coverage mà không thử nhánh false).

Path Coverage

Định nghĩa

Path Coverage: yêu cầu thực thi *mọi đường đi* có thể có từ đầu đến cuối chương trình qua CFG.

- Chặt chẽ nhất trong 3 loại coverage, nhưng số đường đi có thể tăng theo cấp số mũ với vòng lặp/điều kiện lồng nhau
- Thường không khả thi với chương trình lớn – đây là lý do cần các độ đo coverage yếu hơn nhưng thực tế hơn

Cyclomatic Complexity (McCabe)

$$V(G) = E - N + 2$$

trong đó E = số cạnh, N = số nút trong CFG.

Ví dụ

Với hàm `classify`: $N = 5$, $E = 4 \Rightarrow V(G) = 4 - 5 + 2 = 1 +$ số nhánh rẽ $= 3$ (2 điều kiện rẽ nhánh + 1).

Cyclomatic complexity ước lượng *số đường đi độc lập tối thiểu* cần kiểm thử để đạt path coverage cơ bản – cũng là chỉ số phổ biến đo độ phức tạp mã nguồn.

Định nghĩa

Mutation Testing: đánh giá *chất lượng bộ test* bằng cách tạo ra các phiên bản chương trình bị "đột biến" nhỏ (mutant) và kiểm tra xem bộ test có phát hiện ra sự khác biệt không.

Ví dụ

Đột biến: đổi `x < 0` thành `x <= 0`. Nếu bộ test hiện tại vẫn "pass" với mutant này, nghĩa là bộ test chưa đủ mạnh để phát hiện lỗi loại off-by-one.

Test Oracle Problem

Định nghĩa

Test Oracle: cơ chế xác định kết quả mong đợi của một test case là gì.

- Với hệ thống phức tạp, đôi khi *không có* oracle rõ ràng (ví dụ: hệ thống AI, hệ thống mô phỏng)
- Giải pháp: dùng *metamorphic testing* (kiểm tra quan hệ giữa các đầu ra thay vì giá trị tuyệt đối), hoặc so sánh với phiên bản tham chiếu (golden reference)

Test Management: Test Plan

Một test plan tối thiểu cần có:

- Phạm vi kiểm thử (scope) – những gì được và không được kiểm thử
- Chiến lược kiểm thử (test levels, techniques sẽ dùng)
- Tiêu chí vào/ra (entry/exit criteria)
- Lịch trình, nguồn lực, rủi ro của bản thân kế hoạch kiểm thử

Entry Criteria và Exit Criteria

Entry Criteria	Điều kiện cần có trước khi bắt đầu kiểm thử (vd: mã nguồn build thành công, môi trường test sẵn sàng)
Exit Criteria	Điều kiện để coi kiểm thử hoàn thành (vd: đạt X% coverage, không còn lỗi mức Critical/High mở)

Regression Test Selection

Khi codebase lớn, chạy lại toàn bộ test sau mỗi thay đổi có thể rất tốn thời gian:

- **Retest-all**: chạy lại toàn bộ (an toàn nhất, chậm nhất)
- **Test selection**: chỉ chạy các test liên quan tới phần mã đã thay đổi (dựa trên phân tích phụ thuộc)
- **Test prioritization**: sắp xếp thứ tự chạy test theo khả năng phát hiện lỗi cao trước

Bài tập: Tính Cyclomatic Complexity

Cho đoạn mã sau, hãy vẽ CFG và tính $V(G)$:

```
int f(int a, int b) {
    if (a > 0) {
        if (b > 0) return 1;
        else return 2;
    }
    return 0;
}
```

Đáp án: Cyclomatic Complexity

- Có 2 điều kiện rẽ nhánh ($a > 0$ và $b > 0$)
- $V(G) =$ số điều kiện rẽ nhánh $+ 1 = 2 + 1 = 3$
- Cần tối thiểu 3 test case độc lập để đạt path coverage cơ bản: $(a \leq 0)$, $(a > 0, b > 0)$, $(a > 0, b \leq 0)$

Bài tập: Statement vs Branch Coverage

Với hàm `f(a,b)` ở slide trước, cho bộ test chỉ gồm 1 test case: $a = 5, b = 3$.

1. Tính statement coverage
2. Tính branch coverage
3. Đề xuất test case bổ sung để đạt 100% branch coverage

Đáp án: Statement vs Branch Coverage

- Với $a = 5, b = 3$: chỉ đi qua nhánh `return 1` $\Rightarrow$ statement coverage = 3/4 câu lệnh (thiếu `return 2` và `return 0`)
- Branch coverage: mới bao phủ nhánh true của cả 2 điều kiện $\Rightarrow$ 2/4 nhánh = 50%
- Bổ sung: $a = 5, b = -1$ (nhánh false của $b > 0$) và $a = -1$ (nhánh false của $a > 0$) để đạt 100%

Liên hệ White-box Testing với bảo mật

- Coverage cao giúp tăng khả năng phát hiện các đường thực thi hiếm gặp – nơi lỗ hổng bảo mật thường ẩn náu
- Tuy nhiên: 100% coverage **không đảm bảo** không có lỗ hổng (coverage đo “đã chạy qua”, không đo “đã kiểm tra đúng mọi giá trị dữ liệu”)
- Đây là động lực cho kỹ thuật *fuzzing* có hướng dẫn bởi coverage (coverage-guided fuzzing) – học chi tiết ở Chương 4

Condition Coverage và MC/DC

Condition Coverage	Mỗi điều kiện con (trong biểu thức logic phức hợp) phải nhận cả giá trị true và false
MC/DC	*Modified Condition/Decision Coverage*: mỗi điều kiện con phải *độc lập* ảnh hưởng tới kết quả quyết định – tiêu chuẩn bắt buộc trong phần mềm hàng không (DO-178C)

Ví dụ MC/DC

```
if (a && b) {   // Quyet dinh phu thuoc 2 dieu kien: a, b
    ...
}
```

Với branch coverage, 2 test case ($a = T, b = T$ và $a = F, b = F$) đã đủ. Với MC/DC, cần thêm test case cho thấy *riêng a* và *riêng b* đều có thể quyết định kết quả độc lập – tổng cộng cần 3–4 test case tùy cấu trúc biểu thức.

Phân tích luồng dữ liệu tĩnh (Data Flow Analysis)

Định nghĩa

Data Flow Analysis: kỹ thuật phân tích tĩnh theo dõi cách giá trị dữ liệu được định nghĩa (define) và sử dụng (use) xuyên suốt chương trình, không cần thực thi.

- Phát hiện biến dùng trước khi khởi tạo (use-before-def)
- Phát hiện biến được gán nhưng không bao giờ dùng (dead assignment)
- Là nền tảng cho công cụ phân tích tĩnh sẽ thực hành ở Chương 3

Taint Analysis (Phân tích lan truyền dữ liệu bẩn)

Định nghĩa

Taint Analysis: theo dõi dữ liệu từ nguồn không tin cậy (source, vd: input người dùng) xem có chảy tới điểm nguy hiểm (sink, vd: câu lệnh SQL) mà không qua bước làm sạch (sanitization) hay không.

Ví dụ

`username` (source, từ HTTP request) chảy trực tiếp vào `cursor.execute(query)` (sink) mà không qua tham số hóa $\Rightarrow$ taint analysis gắn cờ đây là lỗ hổng SQL Injection tiềm ẩn.

Symbolic Execution (giới thiệu)

Định nghĩa

Symbolic Execution: thực thi chương trình với các giá trị đầu vào *tượng trưng* (symbolic) thay vì giá trị cụ thể, thu thập điều kiện đường đi (path condition) dưới dạng công thức logic.

- Mỗi đường thực thi tương ứng với một path condition (vd: $x > 0 \wedge y < 10$)
- Dùng bộ giải SMT để kiểm tra path condition có thể thỏa mãn (satisfiable) hay không $\Rightarrow$ sinh test case cụ thể tự động
- Nội dung này sẽ thực hành chi tiết trong module `ch3_staticsat`

Ví dụ

Với hệ thống tìm đường đi ngắn nhất, khó biết *chính xác* kết quả đúng cho mọi đồ thị lớn. Nhưng ta có thể kiểm tra *quan hệ*: nếu tăng trọng số một cạnh, độ dài đường đi ngắn nhất không thể giảm. Đây là một **metamorphic relation** dùng để kiểm thử mà không cần oracle tuyệt đối.

Kiểm thử bảo mật chuyên biệt: xem trước Chương 4

- **Fuzz Testing**: sinh tự động lượng lớn input (ngẫu nhiên hoặc dẫn hướng bởi coverage) để tìm input gây crash/lỗi
- **Penetration Testing**: mô phỏng tấn công thực tế bởi chuyên gia bảo mật để tìm lỗ hổng khai thác được
- Cả hai đều bổ sung cho testing chức năng truyền thống, tập trung vào *hành vi ngoài đặc tả*

Bài tập: Chọn kỹ thuật thiết kế test phù hợp

Với mỗi tình huống sau, hãy chọn kỹ thuật phù hợp nhất (equivalence partitioning, decision table, state transition, use case):

1. Kiểm tra hệ thống phê duyệt vay vốn với nhiều điều kiện logic kết hợp (thu nhập, điểm tín dụng, lịch sử nợ)
2. Kiểm tra quy trình đặt vé máy bay từ tìm chuyến $\rightarrow$ chọn ghế $\rightarrow$ thanh toán

Đáp án: Chọn kỹ thuật thiết kế test phù hợp

1. **Decision Table Testing** – vì có nhiều điều kiện logic kết hợp cần kiểm tra đầy đủ tổ hợp
2. **Use Case Testing** – vì đây là một quy trình nhiều bước theo kịch bản sử dụng thực tế của người dùng

Bài tập: Thiết kế Decision Table

Chính sách bảo hiểm: từ chối bồi thường nếu (đã quá hạn thanh toán phí) HOẶC (gian lận được phát hiện). Hãy xây dựng decision table đầy đủ (2 điều kiện $\Rightarrow$ 4 tổ hợp).

Đáp án: Thiết kế Decision Table

	TC1	TC2	TC3	TC4
Quá hạn phí?	Đ	Đ	S	S
Gian lận?	Đ	S	Đ	S
Từ chối bồi thường?	Có	Có	Có	Không

Độ đo chất lượng kiểm thử: Defect Density

$$\text{Defect Density} = \frac{\text{Số lỗi phát hiện}}{\text{Kích thước phần mềm (KLOC hoặc số điểm chức năng)}}$$

- Giúp so sánh chất lượng giữa các module/phiên bản
- Defect density cao bất thường ở một module thường gợi ý module đó cần review/test kỹ hơn (liên hệ nguyên lý *defect clustering* đã học ở 1.3)

Test Automation Framework – Tổng quan

- **Unit test framework**: pytest (Python), JUnit (Java), Google Test (C++)
- **UI/E2E automation**: Selenium, Playwright
- **CI/CD integration**: chạy test tự động mỗi khi có commit/pull request (GitHub Actions, Jenkins...)
- Tự động hóa giúp regression testing khả thi ở quy mô lớn, đặc biệt quan trọng khi vá lỗi bảo mật thường xuyên

Tổng kết 1.4

- Bổ sung kỹ thuật black-box: decision table, state transition, use case, experience-based
- White-box: CFG, statement/branch/path coverage, cyclomatic complexity
- Mutation testing đánh giá chất lượng bộ test; test oracle problem là thách thức thực tế
- Test management: test plan, entry/exit criteria, regression test selection

Vì sao cần mô hình hình thức?

- Testing chỉ kiểm tra được *một số* kịch bản cụ thể – không đảm bảo tính đúng đắn cho *mọi* trường hợp
- Để **kiểm chứng hình thức** (formal verification), ta cần biểu diễn hệ thống bằng một **mô hình toán học** chính xác
- **Kripke Structure** là mô hình phổ biến nhất để biểu diễn hành vi hệ thống dưới dạng các trạng thái và chuyển đổi – nền tảng của *model checking* (Chương 3)

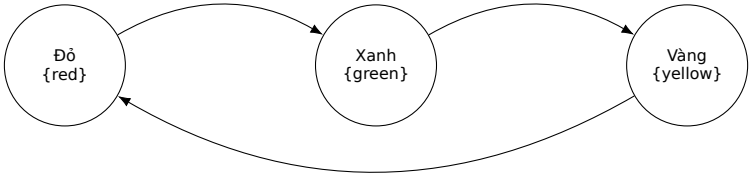
Định nghĩa hình thức Kripke Structure

Định nghĩa

Một **Kripke Structure** là bộ bốn $K = (S, S_0, R, L)$ trong đó:

- S: tập hữu hạn các trạng thái
- $S_0 \subseteq S$: tập trạng thái khởi tạo
- $R \subseteq S \times S$: quan hệ chuyển tiếp (*total*: mọi trạng thái đều có ít nhất một trạng thái kế tiếp)
- $L : S \to 2^{AP}$: hàm nhãn gán cho mỗi trạng thái một tập các mệnh đề nguyên tử (atomic propositions) đúng tại trạng thái đó

Ví dụ đơn giản: Đèn giao thông



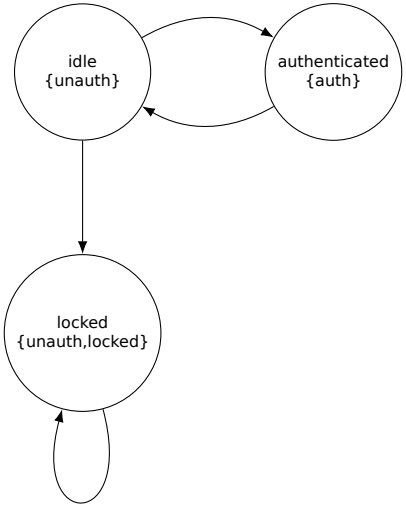
$S = \{$Đỏ, Xanh, Vàng$\}$, $S_0 = \{$Đỏ$\}$, $AP = \{red, green, yellow\}$, mỗi trạng thái được gán đúng một nhãn màu tương ứng.

Ví dụ bảo mật: Mô hình đăng nhập

Quay lại module `ch1_models.kripke.build_login_example()` đã xây dựng:

- $S = \{\text{idle}, \text{authenticated}, \text{locked}\}$
- $S_0 = \{\text{idle}\}$
- $L(\text{idle}) = \{\text{unauth}\}$, $L(\text{authenticated}) = \{\text{auth}\}$, $L(\text{locked}) = \{\text{unauth}, \text{locked}\}$
- R: idle$\to$authenticated, idle$\to$locked, authenticated$\to$idle, locked$\to$locked

Sơ đồ trạng thái mô hình đăng nhập



Đây chính là kết quả xuất TikZ tự động từ hàm `to_tikz()` trong module Python – sinh viên có thể dùng để vẽ mô hình cho hệ thống của riêng mình.

Path (Đường thực thi) trong Kripke Structure

Định nghĩa

Một **path** $\pi = s_0 s_1 s_2 \ldots$ là một dãy trạng thái vô hạn sao cho $s_0 \in S_0$ và $(s_i, s_{i+1}) \in R$ với mọi $i \geq 0$.

Ví dụ

Với mô hình đăng nhập: $\pi =$ idle, authenticated, idle, locked, locked, locked, ... là một path hợp lệ.

Vì R phải *total* (mọi trạng thái có ít nhất 1 kế tiếp), path luôn có thể kéo dài vô hạn – kể cả bằng cách lặp lại self-loop như trạng thái `locked`.

Reachability – Tính đạt đến được

Định nghĩa

Trạng thái s được gọi là **reachable** (đạt đến được) nếu tồn tại một path từ một trạng thái khởi tạo tới s.

Hàm `reachable_states()` trong module Python thực hiện chính xác thuật toán BFS/DFS để tính tập này – đây là bước đầu tiên và cơ bản nhất của mọi kỹ thuật model checking.

Lưu ý

Một thuộc tính bảo mật quan trọng: *"trạng thái nguy hiểm X không bao giờ đạt đến được từ trạng thái khởi tạo"* – đây chính là một dạng **safety property**.

Safety Property vs. Liveness Property

Safety	“Điều xấu không bao giờ xảy ra” – vd: hệ thống không bao giờ vào trạng thái không an toàn
Liveness	“Điều tốt cuối cùng sẽ xảy ra” – vd: mọi yêu cầu cuối cùng sẽ được xử lý

Ví dụ

Safety: “Tài khoản bị khóa không bao giờ tự động mở khóa mà không qua xác minh admin.” Liveness: “Mọi yêu cầu đặt lại mật khẩu cuối cùng sẽ nhận được email phản hồi.”

Counterexample – Phản ví dụ

Định nghĩa

Khi một thuộc tính safety bị vi phạm, **counterexample** là một path cụ thể từ trạng thái khởi tạo dẫn tới trạng thái vi phạm – dùng để gỡ lỗi.

Hàm `find_path(target)` trong module Python chính là công cụ sinh counterexample: nó trả về đường đi cụ thể từ `idle` tới bất kỳ trạng thái nào, ví dụ tới `locked`.

Giới thiệu Temporal Logic (xem trước Chương 3)

Để phát biểu các thuộc tính một cách hình thức, ta cần logic thời gian (temporal logic):

- **LTL** (Linear Temporal Logic): phát biểu thuộc tính trên *từng path* riêng lẻ
- **CTL** (Computation Tree Logic): phát biểu thuộc tính trên *cây các path* có thể có
- Toán tử phổ biến: $\mathbf{G}\varphi$ (luôn luôn φ), $\mathbf{F}\varphi$ (cuối cùng φ), $\mathbf{X}\varphi$ (ở bước kế tiếp φ)

Ví dụ phát biểu thuộc tính bằng LTL

- $\mathbf{G}\neg$danger: “Luôn luôn không ở trạng thái nguy hiểm” (safety)
- $\mathbf{G}$(request $\rightarrow \mathbf{F}$ response): “Mọi yêu cầu cuối cùng đều có phản hồi” (liveness)
- $\mathbf{G}$(locked $\rightarrow \mathbf{X}$ locked $\vee \mathbf{X}$ admin_unlock): “Khi đã khóa, bước tiếp theo vẫn khóa trừ khi admin mở khóa”

Bài toán Model Checking (phát biểu)

Định nghĩa

Model Checking: cho Kripke structure K và công thức temporal logic φ, xác định xem $K \models \varphi$ (K thỏa mãn φ) hay không – một cách *tự động, đầy đủ, và chính xác*.

Nếu $K \not\models \varphi$, thuật toán model checking trả về một **counterexample** cụ thể. Đây là kỹ thuật kiểm chứng có tính *sound* và *complete* đối với mô hình hữu hạn – nội dung cốt lõi của Chương 3.

Bài toán bùng nổ trạng thái (State Explosion)

- Khi hệ thống có *n* biến boolean, số trạng thái có thể lên tới 2^n
- Ví dụ: 30 tiến trình song song, mỗi tiến trình có 10 trạng thái $\Rightarrow 10^{30}$ trạng thái tổ hợp – không thể duyệt hết bằng cách liệt kê tường minh
- Đây là thách thức lớn nhất của model checking thực tế, thúc đẩy các kỹ thuật như symbolic model checking, SAT/SMT-based bounded model checking (Chương 3)

Xây dựng Kripke Structure từ mã nguồn – Ví dụ

```
int counter = 0;
void increment() {
    if (counter < 3) {
        counter++;    // trang thai: counter = 0,1,2,3
    }
}
```

Mỗi giá trị có thể của `counter` (0, 1, 2, 3) tương ứng với một trạng thái trong Kripke structure; lời gọi `increment()` tương ứng với quan hệ chuyển tiếp R.

Bài tập: Xây dựng Kripke Structure

Với đoạn mã `counter` ở slide trước, hãy:

1. Liệt kê tập trạng thái S và trạng thái khởi tạo S_0
2. Vẽ quan hệ chuyển tiếp R
3. Gán nhãn L cho mỗi trạng thái (vd: {zero}, {nonzero}, {max})

Đáp án: Xây dựng Kripke Structure

- $S = \{0, 1, 2, 3\}$, $S_0 = \{0\}$
- R: $0 \to 1, 1 \to 2, 2 \to 3, 3 \to 3$ (self-loop vì điều kiện `counter < 3` sai nên không đổi)
- $L(0) = \{zero\}$, $L(1) = L(2) = \{nonzero\}$, $L(3) = \{nonzero, max\}$

Bài tập thực hành với module Python

Sử dụng `softsec.ch1_models.kripke.KripkeStructure`, hãy:

1. Cài đặt mô hình Kripke cho bài tập `counter` ở trên
2. Gọi `reachable_states()` và kiểm tra kết quả đúng $\{0, 1, 2, 3\}$
3. Gọi `find_path("3")` và giải thích ý nghĩa counterexample trả về

Kripke Structure và bảo mật: tổng kết liên hệ

- Mô hình hóa hệ thống bảo mật bằng Kripke structure cho phép **chứng minh** (không chỉ kiểm thử) các thuộc tính an toàn quan trọng
- Ví dụ ứng dụng thực tế: chứng minh giao thức xác thực không bao giờ để lộ trạng thái “đã xác thực” mà không qua kiểm tra mật khẩu hợp lệ
- Đây là nền tảng lý thuyết cho **Bounded Model Checking** sẽ học chi tiết ở Chương 3 (mục 3.5)

LTL vs. CTL: Path vs. Tree

- **LTL**: công thức được đánh giá trên *một path cụ thể* – phù hợp diễn đạt “theo thời gian, điều gì xảy ra trên một dòng thực thi”
- **CTL**: công thức có *lượng từ trên các path* (**A** – mọi path, **E** – tồn tại path) – phù hợp diễn đạt “từ trạng thái này, có tồn tại cách nào để…”
- Ví dụ CTL: **AG** ¬danger (mọi path, luôn luôn không nguy hiểm) vs. **EF** goal (tồn tại path dẫn tới goal)

Ngữ nghĩa toán tử CTL cơ bản

AGφ	Trên *mọi* path, φ đúng tại *mọi* trạng thái (safety toàn cục)
EFφ	*Tồn tại* một path mà φ đúng tại *một* trạng thái nào đó (reachability)
AFφ	Trên mọi path, cuối cùng φ sẽ đúng (liveness đảm bảo)
EGφ	Tồn tại một path mà φ đúng mãi mãi

Ví dụ

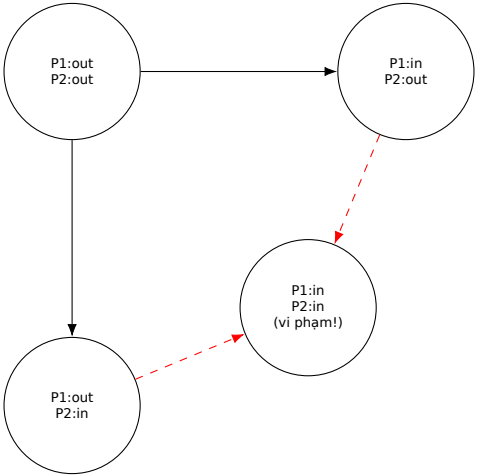
EF locked tương đương với câu hỏi *reachability*: "Trạng thái locked có đạt đến được không?" – chính là điều hàm `is_reachable()` trong module Python trả lời.

Ví dụ kinh điển: Mutual Exclusion (Loại trừ lẫn nhau)

Hai tiến trình cùng truy cập một tài nguyên chia sẻ (critical section):

- Trạng thái: (P1 ở ngoài/trong critical section, P2 ở ngoài/trong critical section)
- Thuộc tính an toàn cần chứng minh: $\mathbf{AG}\,\neg(\text{P1_in} \wedge \text{P2_in})$ – không bao giờ cả 2 cùng ở trong critical section
- Đây là bài toán kinh điển trong hệ điều hành, cũng chính là dạng tổng quát của *race condition* đã học ở mục 1.2

Sơ đồ Kripke Structure cho Mutual Exclusion (rút gọn)



Nếu cơ chế khóa (locking) được thiết kế đúng, trạng thái “vi phạm” (đường nét đứt đỏ) sẽ **không bao giờ** nằm trong tập reachable states.

Bài tập: Kiểm tra thuộc tính bằng tay

Với mô hình đèn giao thông (Đỏ→Xanh→Vàng→Đỏ), hãy kiểm tra bằng tay: công thức $\mathbf{AG}\,\mathbf{EF}\,\text{red}$ (“từ mọi trạng thái, luôn tồn tại đường quay lại đèn đỏ”) có đúng không?

Đáp án: Kiểm tra thuộc tính bằng tay

- Từ Đỏ: có thể tới Xanh, Vàng, rồi quay lại Đỏ $\Rightarrow$ EF red đúng
- Từ Xanh: đi tới Vàng rồi Đỏ $\Rightarrow$ EF red đúng
- Từ Vàng: đi thẳng tới Đỏ $\Rightarrow$ EF red đúng
- Vì đúng tại *mọi* trạng thái $\Rightarrow$ **AG EF** red **đúng** trên mô hình này

Ví dụ khác: Máy bán hàng tự động (Vending Machine)

Trạng thái: `idle` (chờ tiền), `has_coin` (đã nhận đủ tiền), `dispensing` (đang trả hàng).

- R: idle$\rightarrow$has_coin (nhận tiền), has_coin$\rightarrow$dispensing (chọn sản phẩm), dispensing$\rightarrow$idle (hoàn tất)
- Thuộc tính cần đúng: **AG** $(\text{dispensing} \rightarrow \neg\text{has_coin})$ – không thể vừa trả hàng vừa còn giữ trạng thái đã nhận tiền treo (tránh lỗi trả hàng 2 lần cho 1 lần trả tiền)

Bài tập: Reachability trên Vending Machine

Cài đặt mô hình `Vending Machine` ở trên bằng lớp `KripkeStructure` trong module Python, sau đó:

1. Kiểm tra `is_reachable("dispensing")` từ `idle`
2. Thêm một trạng thái lỗi `stuck` không có transition nào dẫn tới, kiểm tra `is_reachable("stuck")` phải trả về `False`

Tổ hợp song song (Parallel Composition) – xem trước

- Hệ thống thực tế thường gồm nhiều thành phần chạy song song (client, server, database...)
- Kripke structure của toàn hệ thống = **tích** (product) của các Kripke structure thành phần
- Đây chính là nguồn gốc chính của *state explosion* – số trạng thái tổ hợp = tích số trạng thái từng thành phần
- Kỹ thuật xử lý (symbolic model checking, bounded model checking) sẽ học ở Chương 3

Bisimulation (giới thiệu ngắn)

Định nghĩa

Hai Kripke structure được gọi là **bisimilar** nếu chúng "trông giống nhau" từ góc độ mọi thuộc tính temporal logic có thể phát biểu – dù số trạng thái có thể khác nhau.

Ý nghĩa thực tế: có thể *rút gọn* một mô hình lớn thành mô hình nhỏ hơn tương đương (bisimilar) để giảm chi phí model checking, mà không làm mất tính chính xác của kết quả kiểm chứng.

Tổng kết 1.5

- Kripke Structure $K = (S, S_0, R, L)$ là mô hình hình thức nền tảng cho model checking
- Path, reachability, safety/liveness property, counterexample là các khái niệm cốt lõi
- Model checking = kiểm tra tự động $K \models \varphi$; thách thức chính là state explosion
- Module Python `ch1_models.kripke` hiện thực hóa các khái niệm này để thực hành

Luyện tập trắc nghiệm (chuẩn bị kiểm tra giữa kỳ) – Câu 1–3

Câu 1. Thuộc tính nào trong CIA bị vi phạm khi một cuộc tấn công DDoS làm sập server?
A. Confidentiality B. Integrity C. Availability D. Authenticity
Câu 2. `strcpy(buffer, input)` không kiểm tra độ dài input thuộc CWE nào?
A. SQL Injection B. Buffer Overflow C. Race Condition D. Path Traversal
Câu 3. Kỹ thuật nào sau đây là *static*?
A. Fuzzing B. Penetration testing C. Static analysis D. Load testing

Đáp án Câu 1–3

1. C (Availability) – DDoS làm hệ thống không sẵn sàng phục vụ.
2. B (Buffer Overflow) – CWE-120/CWE-121.
3. C (Static analysis) – không cần thực thi chương trình.

Luyện tập trắc nghiệm – Câu 4–6

Câu 4. Trong mô hình Kripke $K = (S, S_0, R, L)$, thành phần L có vai trò gì?
A. Tập trạng thái khởi tạo B. Quan hệ chuyển tiếp C. Gán nhãn mệnh đề cho trạng thái D. Tập hành động
Câu 5. Công thức CTL **AG** ¬danger diễn đạt điều gì?
A. Liveness B. Safety C. Reachability D. Fairness
Câu 6. 100% statement coverage có đảm bảo 100% branch coverage không?
A. Luôn đúng B. Luôn sai C. Không nhất thiết D. Chỉ đúng với vòng lặp

Đáp án Câu 4–6

4. C (Gán nhãn mệnh đề cho trạng thái).
5. B (Safety – “luôn luôn không nguy hiểm”).
6. C (Không nhất thiết – ví dụ `if` không có `else`).

Luyện tập trắc nghiệm – Câu 7–9

Câu 7. Nguyên tắc “mỗi thành phần chỉ nên có quyền tối thiểu cần thiết” gọi là gì?
A. Defense in Depth B. Least Privilege C. Fail Secure D. Separation of Duties
Câu 8. Race condition xảy ra giữa 2 bước nào?
A. Input và Output B. Time-of-check và Time-of-use C. Compile và Run D. Design và Code
Câu 9. CVSS dùng để làm gì?
A. Phân loại loại lỗi (weakness) B. Chấm điểm mức độ nghiêm trọng của lỗ hổng C. Đo code coverage D. Quản lý phiên bản phần mềm

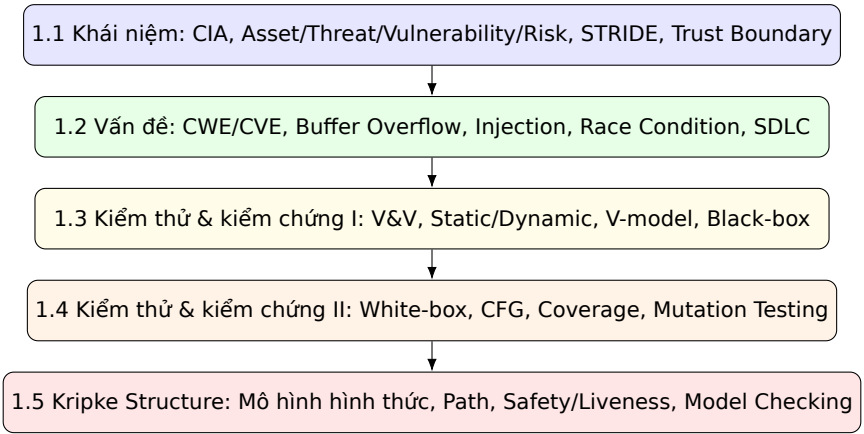
Đáp án Câu 7–9

7. B (Least Privilege).
8. B (Time-of-check to time-of-use – TOCTOU).
9. B (Chấm điểm mức độ nghiêm trọng của lỗ hổng, thang 0–10).

Bảng công thức cần nhớ

Khái niệm	**Công thức**
Risk	Likelihood $\times$ Impact
Statement Coverage	(Số câu lệnh đã chạy) / (Tổng câu lệnh)
Branch Coverage	(Số nhánh đã chạy) / (Tổng số nhánh)
Cyclomatic Complexity	$V(G) = E - N + 2$
Defect Density	(Số lỗi) / (Kích thước phần mềm)

Bản đồ tri thức Chương 1



Ứng dụng thực tế: Kripke Structure trong giao thức bảo mật

- Các giao thức xác thực (vd: TLS handshake) có thể mô hình hóa dưới dạng Kripke structure với trạng thái là các bước bắt tay (hello, key exchange, finished...)
- Công cụ kiểm chứng hình thức chuyên dụng (ProVerif, Tamarin) dùng chính nguyên lý này để *chứng minh* giao thức không có kẽ hở logic cho kẻ tấn công trung gian (man-in-the-middle)
- Đây là ứng dụng công nghiệp thực tế của những gì vừa học, vượt ra ngoài phạm vi học phần nhưng đáng để biết

So sánh Testing vs. Model Checking

	Testing	**Model Checking**
Đối tượng	Chương trình thực thi được	Mô hình trừu tượng hữu hạn
Độ bao phủ	Một số trường hợp cụ thể	Toàn bộ trạng thái mô hình
Kết quả khi sai	Test case gây lỗi	Counterexample (đường đi cụ thể)
Giới hạn	Không đầy đủ	State explosion

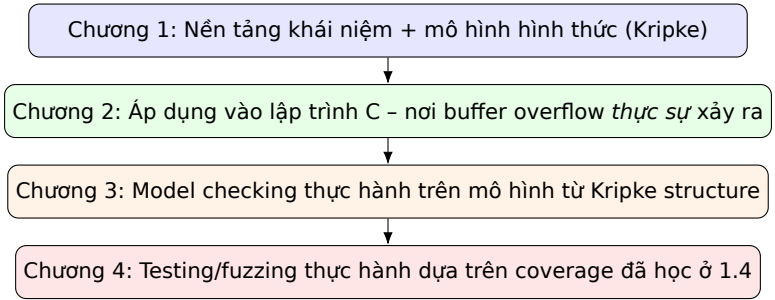
Bài tập tự luận ngắn (ôn tập)

Hãy viết 3–5 câu giải thích: *“Vì sao một hệ thống có thể vượt qua toàn bộ test case chức năng nhưng vẫn chứa lỗ hổng bảo mật nghiêm trọng?”* Liên hệ tới ít nhất 2 khái niệm đã học trong Chương 1.

Gợi ý đáp án bài tự luận

- Test case chức năng thường tập trung vào input “điển hình” theo đặc tả, trong khi lỗ hổng bảo mật thường nằm ở hành vi **ngoài đặc tả** (input bất thường, đường thực thi hiếm gặp)
- Coverage 100% (statement/branch) không đảm bảo mọi *giá trị dữ liệu* nguy hiểm đã được thử – liên hệ tới buffer overflow chỉ xảy ra với input rất dài
- Cần bổ sung: phân tích tĩnh, model checking, fuzzing – các kỹ thuật học ở Chương 3, 4

Liên hệ Chương 1 với toàn bộ học phần



Bảng thuật ngữ cốt lõi (1/2)

CIA	Confidentiality, Integrity, Availability – 3 thuộc tính bảo mật cốt lõi
Threat/Vulnerability/Risk	Mối đe dọa / Điểm yếu / Rủi ro (Likelihood $\times$ Impact)
CWE/CVE	Danh mục loại lỗi / Danh mục lỗ hổng cụ thể
Buffer Overflow	Ghi dữ liệu vượt quá vùng nhớ cấp phát
V&V	Verification (đúng theo đặc tả) và Validation (đúng nhu cầu người dùng)

Bảng thuật ngữ cốt lõi (2/2)

Black-box / White-box	Kiểm thử theo đặc tả / theo cấu trúc mã nguồn nội bộ
Coverage (statement/branch/path)	Độ đo tỷ lệ mã nguồn đã được thực thi qua bộ test
Kripke Structure	$K = (S, S_0, R, L)$ – mô hình trạng thái hình thức
Safety / Liveness	"Điều xấu không xảy ra" / "Điều tốt cuối cùng xảy ra"
Model Checking	Kiểm tra tự động $K \models \varphi$, trả về counterexample nếu sai

Vai trò của Chương 1 trong bài tập lớn

Bài tập lớn cuối kỳ sẽ yêu cầu sinh viên:

- Mô hình hóa một hệ thống nhỏ (do sinh viên tự chọn) dưới dạng Kripke structure
- Thiết kế bộ test case theo cả kỹ thuật black-box và white-box đã học
- Sử dụng công cụ phân tích tĩnh/động (Chương 3, 4) để phát hiện lỗ hổng cụ thể
- Viết báo cáo trình bày rõ ràng: mô hình, test case, lỗ hổng phát hiện, và cách khắc phục

Nguồn tài liệu bổ sung

- Clarke, Grumberg, Peled, *Model Checking* (MIT Press) – tài liệu kinh điển về Kripke structure và temporal logic
- ISTQB Foundation Level Syllabus – chi tiết các kỹ thuật testing đã học
- MITRE CWE Top 25 Most Dangerous Software Weaknesses (cập nhật hàng năm)

Câu hỏi ôn tập tổng hợp (1/3)

1. Phân biệt Security, Safety và Reliability, cho ví dụ minh họa
2. Vì sao `strcpy` bị coi là hàm nguy hiểm? Đề xuất hàm thay thế an toàn hơn
3. Testing có thể chứng minh phần mềm không có lỗi hay không? Giải thích

Câu hỏi ôn tập tổng hợp (2/3)

4. Tính cyclomatic complexity cho một hàm có 3 câu lệnh `if` độc lập (không lồng nhau)
5. Trình bày sự khác biệt giữa statement coverage và branch coverage bằng ví dụ
6. Định nghĩa Kripke Structure và giải thích ý nghĩa từng thành phần

Câu hỏi ôn tập tổng hợp (3/3)

7. Cho một Kripke structure đơn giản, xác định tập trạng thái đạt đến được
8. Phân biệt safety property và liveness property, mỗi loại cho một ví dụ về bảo mật
9. Vì sao state explosion là thách thức chính của model checking?

Đáp án gợi ý câu 1–3

1. Security chống tác nhân chủ đích; Safety chống rủi ro ngẫu nhiên; Reliability là hoạt động đúng liên tục. Ví dụ: xe tự lái có thể an toàn vận hành nhưng bị hack (mất an toàn bảo mật)
2. `strcpy` không kiểm tra độ dài đích $\Rightarrow$ tràn bộ đệm; nên dùng `strncpy` kèm giới hạn độ dài và đảm bảo null-terminator
3. Không – testing chỉ chứng minh *sự tồn tại* của lỗi qua các trường hợp cụ thể đã thử, không thể bao phủ mọi đầu vào có thể có (trừ khi input gian miền hữu hạn nhỏ)

Đáp án gợi ý câu 4–6

4. $V(G) = 3 + 1 = 4$ (3 điều kiện rẽ nhánh độc lập)
5. Statement coverage chỉ yêu cầu mỗi dòng lệnh chạy qua ít nhất 1 lần; branch coverage yêu cầu cả 2 nhánh true/false của mỗi điều kiện đều được thực thi – ví dụ `if(x>0) y=1;` chạy với x=5 đạt 100% statement nhưng chỉ 50% branch coverage
6. $K = (S, S_0, R, L)$: S tập trạng thái, S_0 trạng thái khởi tạo, R quan hệ chuyển tiếp, L hàm gán nhãn mệnh đề đúng tại mỗi trạng thái

Đáp án gợi ý câu 7–9

7. Dùng BFS/DFS từ S_0 theo quan hệ R (như hàm `reachable_states()` trong module Python)
8. Safety: “tài khoản không tự mở khóa khi chưa xác minh”. Liveness: “mọi yêu cầu đặt lại mật khẩu cuối cùng được xử lý”
9. Số trạng thái tổ hợp tăng theo cấp số mũ với số biến/tiến trình song song, khiến việc liệt kê tường minh trở nên bất khả thi với hệ thống thực tế lớn

Bài tập tổng hợp lớn (chuẩn bị kiểm tra giữa kỳ)

Cho hệ thống ATM đơn giản với các trạng thái: `no_card`, `card_inserted`, `pin_verified`, `transaction`, `card_ejected`.

1. Xây dựng mô hình Kripke đầy đủ (S, S0, R, L)
2. Xác định 2 lỗ hổng bảo mật tiềm ẩn theo STRIDE nếu thiết kế sai (vd: chuyển thẳng từ `card_inserted` sang `transaction` mà bỏ qua `pin_verified`)
3. Thiết kế 3 test case theo state transition testing để kiểm tra các lỗ hổng đó

Gợi ý đáp án bài tập tổng hợp

- $S_0 = \{\text{no_card}\}$; R hợp lệ:
  no_card→card_inserted→pin_verified→transaction→card_ejected→no_card
- Lỗ hổng STRIDE: **Elevation of Privilege** nếu bỏ qua bước `pin_verified`; **Tampering** nếu trạng thái `transaction` có thể bị truy cập trực tiếp qua API mà không qua state machine
- Test case: (1) thử truy cập `transaction` khi đang ở `card_inserted` – kỳ vọng bị từ chối; (2) kiểm tra không tồn tại cạnh trực tiếp trong R giữa 2 trạng thái đó; (3) fuzz các chuyển đổi trạng thái không hợp lệ (chi tiết ở Chương 4)

Checklist tự đánh giá cuối chương

Sinh viên nên tự tin trả lời **Có** cho các câu sau trước khi sang Chương 2:

- ☐ Tôi phân biệt được Security/Safety/Reliability và các thuộc tính CIA
- ☐ Tôi có thể phân loại một đoạn mã lỗi vào đúng nhóm CWE
- ☐ Tôi phân biệt được static/dynamic, black-box/white-box, và các loại coverage
- ☐ Tôi có thể xây dựng một Kripke structure đơn giản từ mô tả hệ thống
- ☐ Tôi hiểu vì sao model checking cần đối mặt với state explosion

Cập nhật dự án Python cuối Chương 1

Đến cuối Chương 1, module `ch1_models` đã hoàn chỉnh với:

- `kripke.py`: `KripkeStructure`, reachability, counterexample path, xuất TikZ
- `test_design.py`: sinh equivalence classes và boundary values tự động
- `risk.py`: tính điểm rủi ro và phân loại mức độ
- 15 unit test (pytest) bao phủ toàn bộ 3 module trên

Đây là nền móng: Chương 2 sẽ bổ sung module `ch2_memsafe` để phát hiện lỗi bộ nhớ trên mã C.

Chuẩn bị cho Chương 2

Chương 2 – Lập trình C an toàn: Quản lý bộ nhớ sẽ trình bày:

- 2.1 Đánh giá rủi ro trong lập trình C
- 2.2 Cấu trúc dữ liệu động (con trỏ, heap, linked list...)
- 2.3 Lập trình C an toàn (secure coding)
- 2.4 Sắp xếp cấu trúc dữ liệu (data structure alignment)
- 2.5 Công cụ phân tích tĩnh tìm lỗi bảo mật

Đây là nơi các khái niệm buffer overflow, use-after-free đã giới thiệu ở mục 1.2 sẽ được phân tích sâu và thực hành trực tiếp.

Danh sách kỹ năng thực hành đã đạt được

Sau Chương 1, sinh viên đã có thể:

- Cài đặt và sử dụng lớp `KripkeStructure` bằng Python
- Tự động sinh test case theo equivalence partitioning / boundary value analysis
- Tính toán risk score, cyclomatic complexity, và các độ đo coverage thủ công
- Đọc hiểu và viết công thức temporal logic (LTL/CTL) cơ bản

Điểm cần lưu ý khi ôn tập

- Kiểm tra giữa kỳ là trắc nghiệm trên máy tính – ưu tiên hiểu *khái niệm và phân biệt* hơn là học thuộc lòng định nghĩa
- Các câu hỏi thường yêu cầu *áp dụng* công thức (risk score, cyclomatic complexity) hoặc *phân loại* tình huống (STRIDE, CWE, static/dynamic)
- Luyện tập lại toàn bộ bài tập có đáp án trong 2 phần của Chương 1 trước khi thi

Hỏi đáp

Câu hỏi?

Hoàn thành Chương 1: Giới thiệu về an toàn phần mềm

Hẹn gặp lại ở Chương 2: Lập trình C an toàn – Quản lý bộ nhớ